프로그래밍I - 관계기반

Chap #5 – 동적표를 이용한 알고리즘 설계

문제9 – 분할 수(Partitioned)

partitioned

같은 크기의 정사각형 종이가 n장 있다.

이 종이들을 밑변을 평행하게 연결하여 몇 개를 나열했다. 맨 아래에 나열한 변의 길이는 바로 윗 변의 길이보다 같거나 길어야 한다.

예를 들어 n=5일 경우는 다음과 같은 방법으로 배치가 가능하다.



위 그림을 숫자로 다음과 같이 표현할 수 있다.

(5) (4,1) (3,2) (3,1,1) (2,2,1) (2,1,1,1) (1,1,1,1,1)

n이 입력될 때 가능한 배치를 모두 구하시오. 단 숫자형태로 표현했을 때, 사전순으로 내림차순으로 한 줄에 하나씩 출력하시오.

[입력]
자연수 n(1<=n<30)

입력 예	출력 예
5	5 4 1 3 2 3 1 1 2 2 1 2 1 1 1 1 1 1 1 1

문제9 – 분할 수(Partitioned)

[#1: 분할 정복 - k개로 분할]

1. f(n, k) 함수 정의

   : k 이하의 자연수의 합으로 n을 만들 수 있는 경우의 수

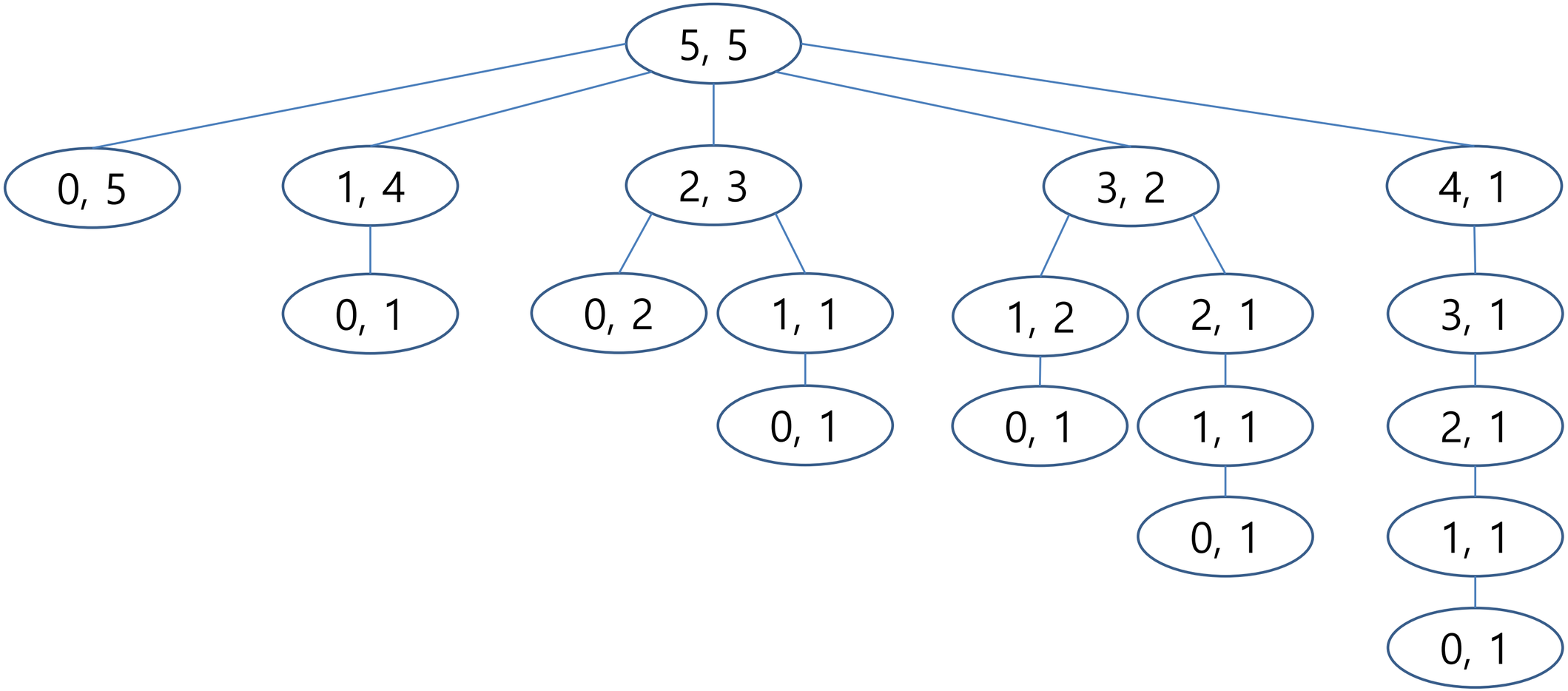
2. 관계식

   $f(n, k) = f(n-1, 1) + f(n-2, 2) + f(n-2, 2) + \dots + f(n-k, k)$

   (단, n>=k)

- 교재 p.68~70

문제9 – 분할 수(Partitioned)



문제9 – 분할 수(Partitioned)

- S(5, 5)
  - a[0]=5, S(0, 5) => print: 5
  - a[0]=4, S(1, 4)
    - a[1]=1, S(0, 1) => print: 4, 1
  - a[0]=3, S(2, 3)
    - a[1]=2, S(0, 2) => print: 3, 2
    - a[1]=1, S(1, 1)
      - a[2]=1, S(0, 1) => print: 3, 1, 1
  - a[0]=2, S(3, 2)
    - a[1]=2, S(1, 2)
      - a[2]=1, S(0, 1) => print: 2, 2, 1
    - a[1]=1, S(2, 1)
      - a[2]=1, S(1, 1)
        - a[3]=1, S(0, 1) => print: 2, 1, 1, 1

- a[0]=1, S(4, 1)
  - a[1]=1, S(3, 1)
    - a[2]=1, S(2, 1)
      - a[3]=1, S(1, 1)
        - a[4]=1, S(0, 1) => print: 1, 1, 1, 1, 1

문제9 – 분할 수(Partitioned)

[#2: 분할 정복 - 2개로 분할(경우의 수만 계산할 경우)]

1. f(n, k) 함수 정의

   : k 이하의 자연수의 합으로 n을 만들 수 있는 경우의 수

2. 관계식

   f(n, k) = f(n-1, 1) + f(n-2, 2) + f(n-2, 2) + ... + f(n-k, k)

   => f(n, k-1) + f(n-k, k) (단, n>=k)

- 교재 p.68~70

동적표(Dynamic Table)란?

- 표의 일부를 채운 후 나머지 칸은 주변의 값을 참조하여 채우는 표
- 첫째 칸: D[1]

1	2	3	4	5	6	7	8	9	10
1									

- i번째 칸: D[i]=D[i-1]+i (동적) or i(i+1)/2 (정적)

1	2	3	4	5	6	7	8	9	10
1	3	6	10	15	21	28	36	45	55

- DT 장점
  : 중복 계산을 제거해 알고리즘의 계산량을 획기적으로 줄일 수 있음
- 교재 p.71~

예제> 피보나치(단순 재귀 vs DT)

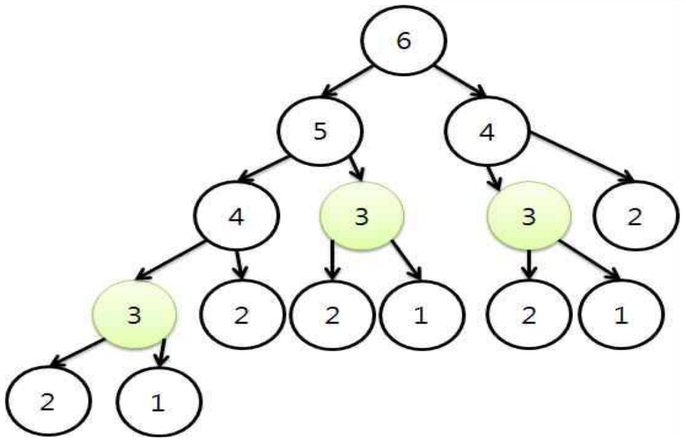
- Q> n을 입력 받아 n 번째 피보나치 수를 출력 (n<=100) (%100009)
- 관계식: $f(n) = \begin{cases} 1 & (n<=2) \\ f(n-1) + f(n-2) & (n>2) \end{cases}$
- 성능 비교:

  #1: 단순 재귀로만 구현할 경우

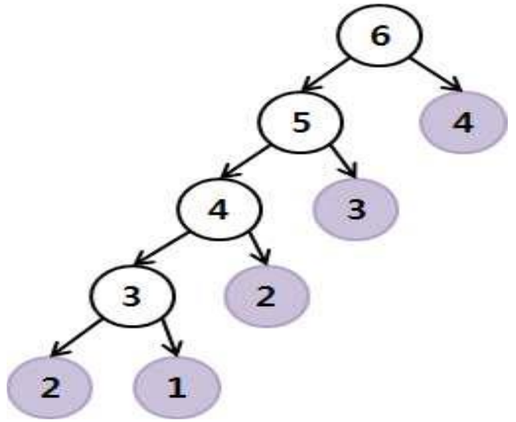
  #2: 동적 테이블(DT)를 적용할 경우 (재귀 함수 호출의 횟수도 비교! N=40)
  DT[n] = n번째 피보나치 수
- 메모이제이션(memoization): 중복 호출을 방지하기 위해 동적표에 데이터를 기록해 두고 재사용 하는 기법

예제> 피보나치(단순 재귀 vs DT)

입력값	실행시간(초)	입력값	실행시간(초)
40	0.435	48	19.163
41	0.685	49	31.064
42	1.083	50	50.210
43	1.739	51	80.838
44	2.827	52	129.340
45	4.551	53	209.532
46	7.272	:	
47	11.888	100	대략 26,241 년



입력값	실행시간(초)	입력값	실행시간(초)
40	0.000	48	0.000
41	0.000	49	0.000
42	0.000	50	0.000
43	0.000	51	0.000
44	0.000		
45	0.000	100	0.000
46	0.000	:	
47	0.000	50,000	0.002



가. 하향식(Top-down) 설계

- 위 ⇒ 아래, n부터 1의 방향으로 해를 구하는 방식, f(n)으로 호출

예1>	예2>

int f(int n) {
  if(n==1) return 1;
  return f(n-1)+n;
}
	int f(int n) {
	  if(n==1) return 1;
	  return 2*f(n/2)+((n+1)/2)*((n+1)/2);
	}

int f(int n) {
  if(n==1) return 1;
  if(!DT[n]) DT[n]=f(n-1)+n;
  return DT[n];
}
	int f(int n) {
	  if(n==1) return 1;
	  if(!DT[n]) DT[n]=2*f(n/2)+((n+1)/2)*((n+1)/2);
	  return DT[n];
	}

나. 상향식(Bottom-up) 설계

- 아래 ⇒ 위, 1부터 n의 방향으로 해를 구하는 방식, f(0) | f(1)으로 호출

예1> 반복문 사용

```
for(int i=1; i<=n; i++) {
    if(i==1)
        D[i]=1
    else
        D[i]=D[i-1]+i;
}
```

예2> 재귀 사용

```
int f(int k) {
    if(k==n+1)
        return 0;
    return k+f(k+1);
    // return D[k]=k+f(k+1);
}
```

문제1> 숫자 뒤집기(L)

[방법#1] DT[n]: n을 거꾸로 출력한 수 => 반복적 방법(상향식) : 효율 저하

$$DT[n] = \begin{cases} n & (n<10) \\ DT[n/10] + (n\%10) * 10^{[\log n]} & (n>=10) \end{cases}$$

[방법#2] DT[n]: f(n)의 값 => DT 적용한 재귀(하향식)

$$f(n) = \begin{cases} n & (n<10) \\ f(n/10) + (n\%10) * 10^{[\log n]} & (n>=10) \end{cases}$$

- 이 문제의 경우 재귀 함수의 중복 호출이 발생하지 않는 구조로 DT 사용이 효율 개선에 도움이 안됨
  => 비선형, 중복 호출이 다수 발생하는 경우 사용하는 것이 적합
- 교재 p. 83~86

문제2> Combination(L)

[방법#1] DT 적용한 재귀(하향식), DT[n][k]=f(n, k)의 값

$$f(n,k) = \begin{cases} 1 & (k=n) \\ n & (k=1) \\ f(n-1,k-1) + f(n-1,k) & (1<k<n) \end{cases}$$

[방법#2] 반복적 방법(상향식), DT[n][k]=n개 중 k개를 고르는 경우의 수

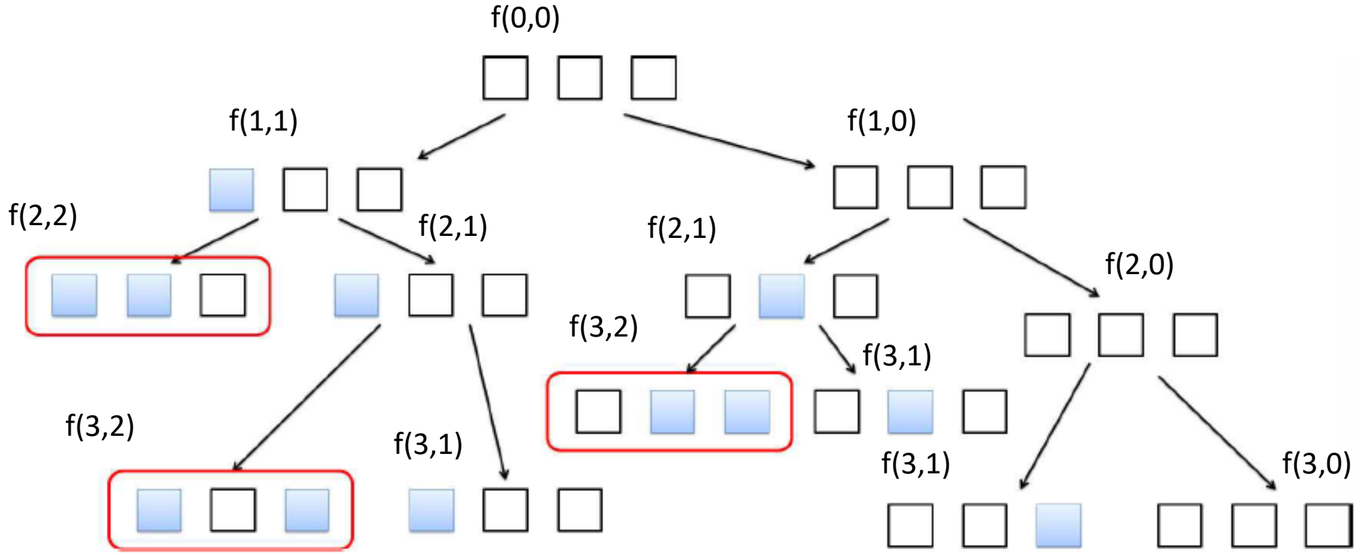
$$DT[n][k] = \begin{cases} 1 & (k=n) \\ n & (k=1) \\ DT[n-1][k-1] + DT[n-1][k] & (1<k<n) \end{cases}$$

- 교재 p. 87~96

문제2> Combination(L)

[방법#3] 전체 탐색법 활용한 상향식 설계(Backtracking)

- f(n, k): 현재까지 k개의 물건을 고른 상태이고,
  n번째 물건을 고를지 말지 결정하고자 하는 상태
- 종료 조건: 선택 성공(k=K), 선택 실패(n=N & k≠K), (단, N, K는 입력 값)
- f(n, k)의 다음 상태: f(n+1, k+1):선택, f(n+1, k): 미선택
- 속도가 느림



문제2> Combination(L)

[방법#4] 다이나미컬 백트래킹(Dynamical Backtracking)

=> 설계는 백트레킹, 효율은 다이나믹(p.95)

(but! 방법1보다 함수 호출의 횟수가 많아 속도는 느림)

- 블록 채우기 문제도 동일한 방법으로 해결 가능!

동적표의 변화 (Dynamic vs Backtrack)

Dynamic(#2)

```
10 7
120
cnt=37
     0     0     0     0     0     0     0
     2     1     0     0     0     0     0
     3     3     1     0     0     0     0
     4     6     4     1     0     0     0
     0    10    10     5     1     0     0
     0     0    20    15     6     1     0
     0     0     0    35    21     7     1
     0     0     0     0    56    28     8
     0     0     0     0     0    84    36
     0     0     0     0     0     0   120
```

Backtrack(#4)

```
10 7
120
cnt=297
   120     0     0     0     0     0     0
    36    84     0     0     0     0     0
     8    28    56     0     0     0     0
     1     7    21    35     0     0     0
     0     1     6    15    20     0     0
     0     0     1     5    10    10     0
     0     0     0     1     4     6     4
     0     0     0     0     1     3     3
     0     0     0     0     0     1     2
     0     0     0     0     0     0     1
```

성능 비교(Combination)

- 방법#1~4을 이용하여 경우의 수를 구하는 프로그램을 작성하고, 결과 및 실행 시간을 비교하시오. (%100009)

입력	f(30, 10)			f(40, 10)			f(100, 10)			f(100, 20)			f(100, 30)			f(200, 100)			f(300, 150)		
	결과 값	실행 시간	호출 수	결과 값	실행 시간	호출 수	결과 값	실행 시간	호출 수	결과 값	실행 시간	호출 수	결과 값	실행 시간	호출 수	결과 값	실행 시간	호출 수	결과 값	실행 시간	호출 수
#1	4			8			6			4			9			8			7		
#2	2			4			8			3			4			9			5		
#3	3			2			7			6			1			8			5		
#4	1			5			9			5			0			0			1		
	5			3			6			9			1			4			7		

연습문제를 풀어서 제출하세요!

[SASA-OJ]

- [[대회]-[2026_2학기_프로그래밍I] 관계기반설계 연습문제 #5

- 기한 내에 제출하세요!
- 수업내용이나 과제 작성 과정에서 질문이 있다면 언제든 **질문**하세요!~
- E-mail:cray25@sasa.hs.kr